



**Hochschule
Bonn-Rhein-Sieg**
University of Applied Sciences

Fachbereich Informatik
Department of Computer Science

Exposé

B. Sc. Informatik

Bewertung von AI Coding Agents für Vibe-Coding in der
Unternehmenssoftwareentwicklung: Vergleich und Integrationsstrategie am
Beispiel BuyIn

Von
Cihad Gök

1. Kontext

Künstliche Intelligenz (KI) hat die Landschaft der Softwareentwicklung grundlegend verändert. Sie ermöglicht es Entwicklern, die Codeproduktion zu beschleunigen, die Qualität zu verbessern und Aufgaben zu automatisieren, die zuvor manuell und zeitaufwendig waren. In den letzten Jahren ist eine neue Kategorie KI-gestützter Entwicklungswerkzeuge entstanden, die als AI Coding Agents bezeichnet wird. Beispiele hierfür sind GitHub Copilot, Cursor, Google Cloud Code, Codeium und Plandex. Diese Lösungen unterstützen die Softwareerstellung, indem sie kontextbezogene Codevorschläge liefern, automatisierte Debugging-Hilfen bereitstellen und die Generierung von Dokumentation integrieren.

Viele dieser Werkzeuge basieren auf großen Sprachmodellen (Large Language Models, LLMs), die auf Grundlage von Benutzereingaben Code vorhersagen und generieren. Ihr Nutzen geht jedoch weit über reine Codevervollständigung hinaus. Sie können unklare Anforderungen interpretieren, Architekturansätze vorschlagen und bei der Restrukturierung bestehender Codebasen unterstützen. Diese Fähigkeiten haben zum Konzept des Vibe-Coding geführt, einem neuartigen Entwicklungsansatz, bei dem die KI nicht als passives Werkzeug, sondern als aktiver kollaborativer Partner verstanden wird. Vibe-Coding weist Ähnlichkeiten mit dem klassischen Pair Programming auf, unterscheidet sich jedoch durch seine permanente Verfügbarkeit, Anpassungsfähigkeit und Eignung sowohl für erfahrene Entwickler als auch für Einsteiger.

Das Aufkommen von AI Coding Agents steht im Einklang mit wachsenden Anforderungen in Unternehmen nach schnellerer Softwarebereitstellung, höherer Produktivität und geringeren technischen Schulden. Trotz ihres Potenzials sind diese Technologien noch relativ neu und ihre Anwendung in Unternehmensumgebungen ist bislang nur unzureichend erforscht. Die meisten vorhandenen Studien konzentrieren sich entweder allgemein auf die Produktivität von Entwicklern oder auf die Leistungsfähigkeit einzelner Werkzeuge in isolierten Szenarien. Folglich fehlt ein umfassendes Verständnis darüber, wie AI Coding Agents in strukturierten Unternehmenskontexten mit unterschiedlichen Kompetenzniveaus eingesetzt werden können.

BuyIn, ein Joint Venture von Orange und der Deutschen Telekom, agiert als international tätige Beschaffungsorganisation. Die Prozesse des Unternehmens stützen sich stark auf effiziente digitale Systeme für Lieferantenmanagement, Vertragslebenszyklus-Tracking und Beschaffungsanalysen. Obwohl BuyIn über eigene interne Softwareentwicklungskapazitäten verfügt, liegt ein erhebliches Innovationspotenzial darin, nicht technisches Personal wie Beschaffungsanalysten zu befähigen, repetitive Prozesse zu automatisieren und einfache digitale Werkzeuge ohne tiefgehende Programmierkenntnisse zu erstellen.

Vor diesem Hintergrund könnten AI Coding Agents zwei gleichwertige Rollen übernehmen. Für professionelle Entwickler beschleunigen sie die Projektdurchführung, unterstützen beim Debugging, generieren Testfälle und verbessern die Wartbarkeit von Unternehmenssystemen. Für Nicht Programmierer ermöglichen sie die Erstellung von Automatisierungsskripten,

einfachen Datenverarbeitungstools oder sogar Prototypen, ohne dass umfangreiche technische Kenntnisse erforderlich sind.

Bevor solche Werkzeuge in die Arbeitsabläufe von BuyIn integriert werden können, müssen mehrere Fragen geklärt werden. Wie wirksam sind verschiedene AI Coding Agents im Kontext von Vibe-Coding sowohl für Fachkräfte als auch für Nicht Programmierer. Welche Agenten liefern in einem Unternehmensumfeld die zuverlässigsten, sichersten und am besten wartbaren Ergebnisse. Welche Integrationsherausforderungen ergeben sich insbesondere in Bezug auf Datensicherheit, Compliance und Wartbarkeit. Mit der Beantwortung dieser Fragen soll diese Arbeit ein umfassendes Verständnis des praktischen Nutzens und der Grenzen von AI Coding Agents in Unternehmensumgebungen schaffen, mit BuyIn als Fallstudie.

2. Problemstellung und Ziele

Problemstellung

Der rasche Aufstieg von AI Coding Agents hat in der Softwareentwicklungs-Community sowohl Begeisterung als auch Skepsis ausgelöst. Befürworter betonen ihr Potenzial zur Verbesserung der Produktivität, der Codequalität und der Entwicklererfahrung, während Kritiker auf Risiken in Bezug auf Genauigkeit, Wartbarkeit und Sicherheit hinweisen. Trotz der zunehmenden Verbreitung in Open-Source- und Freelance-Kontexten gibt es bislang nur wenige empirische Belege für ihre Wirksamkeit in Unternehmensumgebungen.

Wesentliche, noch ungelöste Herausforderungen sind:

1. **Mangel an vergleichenden Studien im Unternehmenskontext**

Die meisten bestehenden Bewertungen untersuchen einzelne Werkzeuge, häufig GitHub Copilot, isoliert. Nur wenige Studien führen systematische Vergleiche mehrerer AI Coding Agents durch, die zugleich unternehmensspezifische Anforderungen wie die Integration in bestehende Toolchains, Compliance-Vorgaben und standardisierte Arbeitsabläufe berücksichtigen.

2. **Begrenzte Forschung zu Vibe-Coding**

Während Pair Programming zwischen Menschen gut erforscht ist, bleibt seine KI-gestützte Variante weitgehend unerforscht. Es existiert keine etablierte Methodik, um zu bewerten, wie KI als kollaborativer Partner Teamarbeit, Kreativität oder Wartbarkeit in professionellen Umgebungen beeinflusst.

3. **Unterschiede in den Nutzerkompetenzen**

Professionelle Entwickler und Nicht-Programmierer nutzen AI Coding Agents auf unterschiedliche Weise. Entwickler optimieren in der Regel komplexe Arbeitsabläufe,

während Nicht-Programmierer stärker auf die KI angewiesen sind, um funktionsfähigen Code zu erzeugen, ohne ein tiefes technisches Verständnis zu besitzen. Vergleichende Analysen dieser beiden Gruppen fehlen weitgehend.

4. Sicherheits- und Compliance-Risiken

Unternehmensumgebungen erfordern einen strengen Schutz sensibler Daten. Falsch konfigurierte AI Coding Agents können vertrauliche Informationen preisgeben, etwa durch offengelegte API-Schlüssel, Umgebungsvariablen oder proprietäre Logik. Jede Implementierung muss den Anforderungen der DSGVO sowie internen Sicherheitsrichtlinien entsprechen.

Vor dem Hintergrund dieser Lücken wird diese Arbeit eine vergleichende, kontextbezogene Untersuchung von AI Coding Agents im Unternehmensumfeld vornehmen und BuyIn als Fallstudie nutzen. Ziel ist es, ihren Nutzen für technische und nicht technische Teams zu bewerten und gleichzeitig Sicherheit, Compliance und Wartbarkeit sicherzustellen.

Ziele

Das übergeordnete Ziel dieser Arbeit ist es, die Wirksamkeit, Anwendbarkeit und sicherheitsrelevanten Implikationen von AI Coding Agents innerhalb des Vibe-Coding-Paradigmas zu untersuchen, mit besonderem Fokus auf die Unternehmenssoftwareentwicklung bei BuyIn.

Die spezifischen Ziele lauten:

Vergleichende Bewertung

- Untersuchung mehrerer AI Coding Agents (GitHub Copilot, Cursor, Google Cloud Code, Plandex, Codeium) anhand eines strukturierten, nutzenorientierten Bewertungsrahmens.
- Analyse ihrer Leistung in unterschiedlichen Entwicklungsszenarien:
 - Greenfield Development: Start eines neuen Projekts von Grund auf.
 - Brownfield Development: Erweiterung oder Anpassung bestehender Unternehmens-Codebasen.

Analyse nach Kompetenzstufen

- Untersuchung der Unterschiede in der Wirksamkeit der Werkzeuge für:
 - Professionelle Entwickler, die mit Arbeitsabläufen der Unternehmensentwicklung vertraut sind.
 - Nicht-Programmierer, beispielsweise Mitarbeiter aus dem Beschaffungs- oder Business-Bereich mit geringen oder keinen Programmierkenntnissen.

Überprüfung von Sicherheit und Compliance

- Analyse, wie jedes Werkzeug mit Datensicherheit, API-Schlüsseln und Umgebungsvariablen umgeht.
- Bewertung der Einhaltung der DSGVO sowie interner Datenschutzrichtlinien.
- Identifizierung potenzieller Risiken und Entwicklung geeigneter Gegenmaßnahmen.

Strategie für die Integration in Unternehmensprozesse

- Ermittlung potenzieller Anwendungsfälle innerhalb von BuyIn, in denen AI Coding Agents die Effizienz steigern und Innovation fördern können.
- Erarbeitung von Empfehlungen für die Einführung, einschließlich Leitlinien für Schulung, Governance und sicheren Betrieb.

Beitrag zur wissenschaftlichen Erkenntnis

- Erweiterung der bestehenden Forschung durch empirische, unternehmensbezogene Erkenntnisse zum KI-gestützten Vibe-Coding.
- Entwicklung eines praxisnahen Bewertungsrahmens, der auch von anderen Organisationen genutzt werden kann.

3. Stand der Forschung

Das Aufkommen von AI Coding Agents stellt einen der bedeutendsten Umbrüche in der modernen Softwareentwicklung dar. Während KI-gestützte Codevervollständigung bereits seit mehr als einem Jahrzehnt existiert, zum Beispiel durch IntelliSense in Visual Studio, hat die Einführung von auf großen Sprachmodellen (Large Language Models, LLMs) basierenden Programmierassistenten die Möglichkeiten solcher Werkzeuge erheblich erweitert.

GitHub Copilot, das 2021 veröffentlicht und von OpenAIs Codex angetrieben wurde, war einer der ersten weit verbreiteten KI-basierten Programmierassistenten. Er zeigte, dass KI ganze Codeausschnitte erzeugen, Unit Tests schreiben und Dokumentation auf Grundlage natürlicher Sprache vorschlagen kann. Seitdem sind konkurrierende Werkzeuge wie Cursor, Google Cloud Code, Codeium und Plandex auf den Markt gekommen, die jeweils darauf abzielen, Geschwindigkeit, Genauigkeit und die Entwicklererfahrung weiter zu verbessern.

3.1 Forschung zu AI Coding Agents

Aktuelle wissenschaftliche und industrielle Studien heben mehrere potenzielle Vorteile von KI-gestütztem Programmieren hervor.

Microsoft Research (2023) zeigte, dass Entwickler mit GitHub Copilot Aufgaben um bis zu 55 Prozent schneller erledigen als ohne Unterstützung durch KI.

Die Studie *Copilot Effect* der Stanford HCI Group stellte fest, dass Entwickler weniger mentale Ermüdung verspüren und mehr Zeit für konzeptionelle Designaufgaben haben.

Einige Untersuchungen legen zudem nahe, dass KI-Agenten helfen können, Syntaxfehler zu reduzieren und die Einhaltung bewährter Praktiken zu verbessern, insbesondere bei gut dokumentierten Programmiersprachen.

Die meisten dieser Studien konzentrieren sich jedoch auf Einzelbewertungen bestimmter Werkzeuge und auf individuelle Entwickler, oft unter kontrollierten Laborbedingungen statt in komplexen Unternehmensumgebungen.

3.2 Vibe-Coding: Ein neues Paradigma

Der Begriff Vibe-Coding ist in der Softwareentwicklungs-Community noch relativ neu. Er bezeichnet einen kollaborativen Programmierstil, bei dem eine KI als dauerhafter und reaktionsfähiger Partner agiert. Obwohl er dem klassischen Pair Programming ähnelt, bei dem zwei menschliche Entwickler zusammenarbeiten, ersetzt Vibe-Coding einen der Partner durch ein KI-System, das jederzeit verfügbar ist, sich anpasst und kontextbezogene Vorschläge liefert.

Pair Programming zwischen Menschen ist gut erforscht und dafür bekannt, die Codequalität zu verbessern, den Wissensaustausch zu fördern und Fehlerquoten zu senken. Im Gegensatz dazu sind Modelle der Zusammenarbeit zwischen Mensch und KI bislang weit weniger verstanden, insbesondere hinsichtlich ihrer Auswirkungen auf Teamarbeit, Kreativität und Wartbarkeit von Software.

Zentrale offene Fragen der Vibe-Coding-Forschung sind:

Wie beeinflusst KI Problemlösungsstrategien im Vergleich zu menschlichen Partnern?

Fördert oder hemmt KI die Kreativität im Softwaredesign?

Wie passen Entwickler ihren Arbeitsablauf an, wenn sie mit einer KI zusammenarbeiten?

Derzeit existiert kein standardisiertes Bewertungsmodell für Vibe-Coding, und die wissenschaftliche Literatur in diesem Bereich ist noch sehr begrenzt.

3.3 Unternehmensspezifische Überlegungen

Während Start-ups und einzelne Entwickler KI-gestützte Programmierassistenten oft ohne größere Einschränkungen nutzen können, gelten in Unternehmensumgebungen wie bei BuyIn strengere Anforderungen.

AI Coding Agents müssen sich nahtlos in bestehende Werkzeuge wie Jira, GitHub Enterprise

oder interne CI/CD-Pipelines integrieren lassen.

Sie müssen vertrauliche Unternehmensdaten sicher verarbeiten und den Vorgaben der DSGVO entsprechen.

Zudem muss KI-generierter Code nachvollziehbar, wartbar und überprüfbar sein.

Schließlich bestehen in Unternehmen oft Teams mit sehr unterschiedlichen Kompetenzniveaus, von hochqualifizierten Entwicklern bis hin zu Mitarbeitenden mit geringen Programmierkenntnissen.

Die aktuelle Forschung geht auf diese spezifischen Anforderungen kaum ein. Die bestehende Lücke liegt in vergleichenden, praxisnahen Untersuchungen, die sowohl die technische Leistungsfähigkeit als auch organisatorische Rahmenbedingungen berücksichtigen.

3.4 Forschungslücken

Trotz vielversprechender erster Ergebnisse bestehen deutliche Grenzen im derzeitigen Forschungsstand.

Es fehlen vergleichende Studien, die mehrere AI Coding Agents innerhalb derselben Umgebung untersuchen.

Der Fokus auf Unternehmenskontexte mit strenger Governance, hohen Sicherheitsanforderungen und Integrationsbedarf ist bislang gering.

Es existiert keine systematische Analyse verschiedener Nutzergruppen, also professioneller Entwickler im Vergleich zu Nicht-Programmierern.

Ebenso gibt es nur sehr wenige Untersuchungen, die Vibe-Coding als strukturierte Methodik betrachten.

4. Vorgehensweise und Bewertung

Diese Arbeit verfolgt einen vergleichenden, nutzenorientierten Bewertungsansatz, um die Leistungsfähigkeit, Benutzerfreundlichkeit und Eignung mehrerer AI Coding Agents für den Unternehmenseinsatz im Kontext von Vibe-Coding zu untersuchen. Die Forschung kombiniert eine literaturgestützte Analyse mit praktischen Tests, die sowohl in simulierten als auch in realen Szenarien bei BuyIn durchgeführt werden.

4.1 Auswahl der Werkzeuge

Die Bewertung konzentriert sich auf eine repräsentative Auswahl moderner AI Coding Agents, die sowohl für die Unternehmenssoftwareentwicklung als auch für die individuelle Produktivität relevant sind:

- **GitHub Copilot:** Marktführer, weitgehend integriert in Entwicklungsumgebungen wie Visual Studio Code und JetBrains-Produkte.

- **Cursor:** Entwicklungsumgebung mit starkem Fokus auf KI, fortgeschrittener Kontextwahrnehmung und integrierten Funktionen für konversationales Programmieren.
- **Google Cloud Code:** Unternehmensorientierte KI-Unterstützung für Programmierung mit enger Anbindung an Google-Cloud-Dienste.
- **Plandex:** Agentenbasierter Entwicklungsassistent, optimiert für mehrstufige Programmierabläufe und Projektplanung.
- **Codeium:** Datenschutzorientiertes KI-Tool für Programmierung mit Offline- und Self-Hosting-Optionen.

4.2 Bewertungsrahmen

Die Untersuchung folgt einem nutzenorientierten Analyseansatz, bei dem jedes Werkzeug anhand vordefinierter Kriterien mit quantitativen und qualitativen Methoden bewertet wird.

Bewertungskriterien:

- **Produktivität:** Geschwindigkeit bei der Bearbeitung von Programmieraufgaben, etwa die Zeit, die für die Implementierung einer bestimmten Funktion benötigt wird.
- **Codegenauigkeit:** Qualität und Korrektheit des generierten Codes, einschließlich Anzahl der Fehler und erforderlicher Korrekturen.
- **Integration:** Leichtigkeit der Anbindung des Werkzeugs an BuyIns bestehende Toollandschaft, zum Beispiel Jira, GitHub Enterprise oder CI/CD-Pipelines.
- **Datensicherheit und Compliance:** Sicherer Umgang mit API-Schlüsseln, Umgebungsvariablen und vertraulicher Geschäftslogik unter Einhaltung der DSGVO und interner Richtlinien.
- **Benutzerfreundlichkeit:** Lernkurve und allgemeine Zugänglichkeit sowohl für professionelle Entwickler als auch für Nicht-Programmierer.
- **Kollaborationsfunktionen:** Unterstützung von Vibe-Coding-Workflows, einschließlich kontextbezogener Verlaufsinformationen, Zusammenarbeit mehrerer Nutzer und gemeinsamer KI-Eingaben.

4.3 Testszenarien

Die Bewertung erfolgt anhand von drei zentralen Szenarien:

Greenfield Development

Entwicklung einer kleinen, eigenständigen Anwendung oder eines Moduls von Grund auf. Zweck: Bewertung, wie effektiv die KI Planung, Strukturierung und Implementierung neuer Projekte unterstützt.

Brownfield Development

Erweiterung oder Überarbeitung eines bestehenden Codebestands, der für BuyIns operative

Abläufe relevant ist, zum Beispiel interne Tools oder Automatisierungsskripte.

Zweck: Untersuchung, inwiefern die KI bestehenden Code interpretieren, anpassen und verbessern kann, während sie bestehende Standards einhält.

Anwendungsfall für Nicht-Programmierer

Anleitung einer nicht technischen Person bei der Erstellung eines einfachen Automatisierungsskripts oder eines Basis-Datenanalysetools mit KI-Unterstützung.

Zweck: Bewertung der Zugänglichkeit, Fehlertoleranz und der Qualität der von Nicht-Programmierern erzeugten Ergebnisse.

4.4 Datenerhebung und Analyse

Quantitative Daten

- Zeit, die zur Bearbeitung jeder Aufgabe benötigt wird.
- Anzahl erforderlicher Korrekturen oder Neufassungen.
- Verhältnis relevanter zu irrelevanten Vorschlägen der KI.

Qualitative Daten

- Nutzerfeedback zur Benutzerfreundlichkeit und zum wahrgenommenen Nutzen.
- Beobachtungen zum Zusammenarbeitsverhalten während der Vibe-Coding-Sitzungen.
- Wahrnehmung von Vertrauen und Zuverlässigkeit in KI-generierten Lösungen.

4.5 Sicherheits- und Compliance-Bewertung

Aufgrund der strengen Compliance-Anforderungen bei BuyIn wird jeder AI Coding Agent hinsichtlich folgender Aspekte untersucht:

- Potenzielle Offenlegung vertraulicher Daten durch generierten Code, einschließlich Risiken durch externe API-Aufrufe, unbeabsichtigte Einbindung proprietärer Geschäftslogik oder sogenannte Prompt-Injection-Angriffe.
- Sicheres Management von API-Schlüsseln und Umgebungsvariablen, damit vertrauliche Zugangsdaten weder offengelegt noch unsachgemäß gespeichert werden.
- Verfügbarkeit von On-Premise- oder Self-Hosting-Optionen, um die Abhängigkeit von externen Cloud-Diensten zu minimieren und die Kontrolle über Datenflüsse zu erhöhen.
- Protokollierungs- und Nachverfolgungsfunktionen für von der KI erzeugten Code, um Nachvollziehbarkeit zu gewährleisten, Compliance-Prüfungen zu erleichtern und die Wartbarkeit über den gesamten Software-Lebenszyklus sicherzustellen.

4.6 Erwartetes Ergebnis

Durch die Kombination technischer Leistungskennzahlen, Sicherheitsbewertungen und Rückmeldungen zur Benutzerfreundlichkeit von professionellen Entwicklern und Nicht-Programmierern verfolgt die Arbeit folgende Ziele:

- Ermittlung der am besten geeigneten AI Coding Agents für BuyIns betriebliche und sicherheitsrelevante Anforderungen.
- Bereitstellung eines klaren, evidenzbasierten Entscheidungsrahmens für die Auswahl geeigneter Werkzeuge.
- Entwicklung von Best-Practice-Empfehlungen für die Integration von KI-gestütztem Vibe-Coding in Unternehmensprozesse.

5. Zeitplan

Die geplante Dauer dieser Bachelorarbeit beträgt 13 Wochen. Die Arbeit ist in klar abgegrenzte Phasen gegliedert, um einen systematischen Ablauf von der Literaturrecherche bis zur Abgabe sicherzustellen. Der folgende Zeitplan gibt einen Überblick über die wichtigsten Aktivitäten und Meilensteine.

Woche Aktivitäten

1–2 Projektstart und Literaturrecherche

- Abstimmung des Umfangs und der Zielsetzung mit BuyIn und dem akademischen Betreuer.
- Durchsicht der vorhandenen Literatur zu AI Coding Agents, Vibe-Coding und Methoden des Pair Programming.
- Sichtung von Branchenberichten und Herstellerdokumentationen zu den ausgewählten Tools.
- Festlegung der ersten Bewertungskriterien.

3 Entwicklung des Bewertungsrahmens

- Definition quantitativer und qualitativer Leistungskennzahlen.
- Entwicklung eines Bewertungsmodells für den Vergleich.
- Auswahl der endgültigen AI Coding Agents für die Tests.

- 4–5 Testphase 1 – GitHub Copilot und Cursor**
- Durchführung von Tests in Greenfield- (neues Projekt) und Brownfield-Szenarien (bestehender Code).
 - Erfassung von Produktivitätskennzahlen, Fehlerraten und Feedback zur Benutzerfreundlichkeit.
 - Beginn der Zusammenstellung erster Ergebnisse.
- 6–7 Testphase 2 – Google Cloud Code, Plandex, Codeium**
- Wiederholung der gleichen Testmethodik für diese Werkzeuge.
 - Einbeziehung mindestens eines Nicht-Programmierer-Falls je Tool.
 - Dokumentation qualitativer Beobachtungen während der Vibe-Coding-Sitzungen.
 - **Vorbereitung einer Präsentation für den Professor über den aktuellen Stand der Arbeit.**
- 8 Datenanalyse – Professionelle Entwickler**
- Vergleich der Ergebnisse aus den Tests mit professionellen Entwicklern.
 - Identifizierung von Trends, Stärken und Schwächen der Werkzeuge.
- 9 Datenanalyse – Nicht-Programmierer**
- Analyse der Resultate aus den Tests mit Nicht-Programmierern.
 - Bewertung von Benutzerfreundlichkeit, Zugänglichkeit und Lernkurve.
- 10 Zusammenführung der vergleichenden Analyse**
- Synthese der Erkenntnisse beider Nutzergruppen.
 - Bewertung der Werkzeuge anhand der festgelegten Kriterien.
 - Entwurf von Empfehlungen für die Integration bei BuyIn.
- 11–12 Schreibphase**
- Verfassen aller Kapitel der Arbeit (Einleitung, Literaturübersicht, Methodik, Ergebnisse, Diskussion, Fazit).
 - Einbindung von Diagrammen, Vergleichstabellen und Erkenntnissen aus der Fallstudie.
- 13 Abschluss und Abgabe**
- Korrekturlesen, Formatkontrolle und Fertigstellung der Literaturangaben.
 - Abgabe der Arbeit beim akademischen Betreuer und bei den Stakeholdern von BuyIn.

6. Vorläufiges Inhaltsverzeichnis

1. **Einleitung**
 - 1.1 Hintergrund und Motivation
 - 1.2 Forschungsproblem und Ziele
 - 1.3 Forschungsfragen
 - 1.4 Aufbau der Arbeit
 2. **Theoretischer Rahmen**
 - 2.1 Definition: AI Coding Agents
 - 2.2 Überblick über Large Language Models in der Softwareentwicklung
 - 2.3 Konzept des Vibe-Coding und Beziehung zum Pair Programming
 - 2.4 Relevanz von AI Coding Agents in der Unternehmenssoftwareentwicklung
 3. **Stand der Forschung**
 - 3.1 Marktüberblick zu AI Coding Agents (Copilot, Cursor, Cloud Code, Plandex, Codeium)
 - 3.2 Bestehende Forschung zur KI-gestützten Softwareentwicklung
 - 3.3 Vibe-Coding im Kontext der Mensch-KI-Zusammenarbeit
 - 3.4 Identifizierte Forschungslücken
 4. **Forschungsmethodik**
 - 4.1 Forschungsdesign und Vorgehensweise
 - 4.2 Auswahl der AI Coding Agents für die Bewertung
 - 4.3 Bewertungskriterien und Scoring-Modell
 - 4.4 Testszenarien (Greenfield, Brownfield, Anwendungsfall für Nicht-Programmierer)
 - 4.5 Methoden der Datenerhebung (quantitativ und qualitativ)
 - 4.6 Rahmen für die Bewertung von Sicherheit und Compliance
 5. **Ergebnisse und Analyse**
 - 5.1 Ergebnisse der Bewertung – Professionelle Entwickler
 - 5.2 Ergebnisse der Bewertung – Nicht-Programmierer
 - 5.3 Vergleichende Nutzenanalyse der AI Coding Agents
 - 5.4 Bewertung von Sicherheit und Compliance
 6. **Use-Case-Implementierung bei BuyIn**
 - 6.1 Identifizierte Anwendungsfälle für AI Coding Agents bei BuyIn
 - 6.2 Strategie zur Umsetzung von Vibe-Coding in der Unternehmensentwicklung
 - 6.3 Herausforderungen und Risiken bei der Einführung
 - 6.4 Empfohlene Governance- und Best-Practice-Ansätze
 7. **Diskussion**
 - 7.1 Interpretation der Ergebnisse
 - 7.2 Auswirkungen auf Praktiken der Softwareentwicklung
 - 7.3 Grenzen der Studie
 - 7.4 Chancen für zukünftige Forschung
 8. **Schlussfolgerung und Empfehlungen**
 - 8.1 Zusammenfassung der wichtigsten Ergebnisse
-

8.2 Empfehlungen für BuyIn

8.3 Breitere Implikationen für die Einführung von AI Coding Agents in Unternehmen

9. **Literaturverzeichnis**

10. **Anhänge**

A. Detaillierte Bewertungskriterien

B. Rohdaten aus den Tests

C. Interview- und Beobachtungsnotizen

7. Vorläufige Literatur

Wissenschaftliche Publikationen und Forschungsarbeiten

- Microsoft Research (2023). *Pair Programming with AI Assistants: Productivity and Quality Implications*.
Verfügbar unter: <https://www.microsoft.com/research>
Grundlegende Studie darüber, wie KI-gestütztes Programmieren Produktivität und Codequalität beeinflusst.
 - Ziegler, C., Lee, M. u. a. (2023). *The Copilot Effect: Understanding the Impact of AI on Software Development Workflows*. Stanford HCI Group.
DOI: 10.1145/3544548.3580642
Untersuchung des Verhaltens von Entwicklern bei der Arbeit mit GitHub Copilot.
 - Nadi, S. u. a. (2022). *Human-AI Collaboration in Software Development*. IEEE Software, 39(6), 20–28.
DOI: 10.1109/MS.2022.3203456
Analyse von Mustern und Herausforderungen bei der Integration von KI in professionelle Entwicklungsprozesse.
 - Vaithilingam, P. u. a. (2022). *Expectations, Outcomes, and Challenges of Modern Code Completion Tools*. Proceedings der 44. Internationalen Konferenz für Software Engineering (ICSE).
DOI: 10.1145/3510003.3510134
Erkenntnisse zu Benutzerfreundlichkeit und Vertrauen von Entwicklern in KI-gestützte Programmierwerkzeuge.
-

Branchenberichte und Whitepapers

- GitHub Next (2023). *How AI is Transforming Code Collaboration*. GitHub Whitepaper.
Darstellung aktueller Branchentrends und Strategien für die Einführung von KI-gestützter Softwareentwicklung.
 - Plandex AI (2024). *Enterprise AI Agent Development: Use Cases and Best Practices*.
Fokussiert auf den Einsatz von KI-Agenten als kollaborative Programmierpartner in Unternehmensumgebungen.
 - Google Developers (2023). *Cloud Code Documentation*.
Verfügbar unter: <https://cloud.google.com/code>
-

Technische Dokumentation zu den KI-gestützten Entwicklungsfunktionen von Google Cloud.

- Amazon Web Services (2023). *CodeWhisperer: AI Coding Assistant for the Enterprise*. AWS Whitepaper.
Sicherheits- und Compliance-Aspekte für den Einsatz von KI-gestützten Programmierassistenten im Unternehmen.

Relevante Methodik-Quellen

- Keeney, R.L., Raiffa, H. (1993). *Decisions with Multiple Objectives: Preferences and Value Trade-offs*. Cambridge University Press.
Grundlegende Methodik für nutzenorientierte Bewertungsrahmen.
- Pohl, K., Rupp, C. (2015). *Requirements Engineering: Fundamentals, Principles, and Techniques*. Springer.
Relevante Quelle zur Definition und Strukturierung von Unternehmens-Use-Cases in Softwareprojekten.